STE Mapbook

Technical Guide

Elephant Movescape Analysis — Methodology & Calculation Reference

Version 1.3 · Generated October 23, 2026

1. Overview

The **STE Mapbook** workflow is a data-to-report pipeline purpose-built for analysing African elephant (*Loxodonta africana*) movement ecology. It ingests GPS telemetry data from **EarthRanger**, performs a sequence of spatial and statistical computations, and delivers six map products, summary statistics, an interactive dashboard, and a print-ready Word mapbook — all organised per individual elephant.

The workflow is implemented as an **Ecoscope Workflow** (YAML spec) and executes within the ecoscope-workflows runtime. Each task in the spec is a composable, versioned function; tasks are chained via data references (${{ workflow.<id>.return }}) and the engine resolves execution order automatically.

> **Note:** Although the trajectory segment filter defaults are tuned for elephants (max speed 9 km/h, max gap 6 h), every threshold is user-configurable at run time through the workflow form.

2. Dependencies & Prerequisites

2.1 EarthRanger Connection

All telemetry data is sourced from an **EarthRanger** instance (set_er_connection task). The user must select a pre-configured EarthRanger data source. The workflow calls get_subjectgroup_observations twice — once for the current analysis period and once for the comparison (previous) period.

- The connection is passed as client to every EarthRanger fetch task.
- Authentication is handled by the Ecoscope EarthRanger client; no credentials are stored in the spec.
- The filter: clean parameter discards any observation already flagged as junk in EarthRanger before the data even reaches the workflow.

2.2 Subject Group

The user picks a **Subject Group Name** (default: Elephants) from a drop-down of the subject groups on the chosen EarthRanger data source. The list is filled by the EarthRangerEnumResolver form transformer once a data source is selected, so the name always matches a group that exists. The workflow fetches every observation for all subjects within that group over the configured time range.

> **Note:** If the group contains no observations in the requested period, the workflow skips all downstream tasks gracefully via the any_is_empty_df skipif condition.

Both observation fetches set include_subject_additional: false, so each subject's free-form additional JSON field is not retained. This is fixed in the spec and no longer exposed on the form.

2.3 Google Earth Engine Project

A **Google Earth Engine (GEE)** project is required for the seasonal analysis component. The task custom_determine_season_windows queries NDVI time-series data from GEE to classify the analysis period into wet and dry seasons. The GEE project name is supplied by the user via set_gee_connection.

2.4 Map Overlay

All six maps share a common overlay layer configured through the **Map Overlay** task-group (select_map_overlay). Two overlay sources are available:

- **LandDx** — standard protected-area polygons (community conservancies, national reserves, national parks) downloaded automatically from a Dropbox URL. No local copy is required; the URL is pre-configured by default.
- **EarthRanger Spatial Feature** — fetches a named spatial feature set directly from the connected EarthRanger instance, allowing site-specific boundary layers to be used as the overlay.

When the **LandDx** option is selected, the geodatabase is filtered to three land-use categories:

- Community Conservancy
- National Reserve
- National Park

Only three columns are retained: type, name, and geometry. Each category is then styled independently:

Land-use Type	Fill / Line Colour	Opacity
Community Conservancy	RGB(166, 182, 151) #a6b697	17.5 %
National Reserve	RGB(136, 167, 142) #88a78e	17.5 %
National Park	RGB(17, 86, 49) #115631	17.5 %

A text label layer (centroid-anchored, Arial, 1 000 m base size, 40–75 px clamp) shows protected-area names at appropriate zoom levels.

> **Note:** The Map Overlay task uses skipif: never, so it always runs regardless of whether upstream data is empty. This ensures the overlay is consistently applied to all six map products.

2.5 Base Map Tile Layer

The background raster is set by the **Configure basemap layers** step (the platform's set_base_maps task). It ships pre-filled with the **ArcGIS World Hillshade** tile service, but the URL, opacity, and max zoom are user-configurable rather than hard-coded:

```
https://server.arcgisonline.com/arcgis/rest/services/Elevation/World_Hillshade/MapServer/tile/{z}/{y}/{x}
```

Default opacity is 1.0 (fully opaque) with a max zoom of 20.

3. Data Ingestion Pipeline

3.1 Observations → Relocations

Raw EarthRanger observations are converted to a standardised **relocations GeoDataFrame** via process_relocations. The following columns are extracted and retained:

Column	Source field	Description
groupby_col	internal	Subject identifier used for grouping
fixtime	observation timestamp	UTC datetime of the GPS fix
junk_status	EarthRanger flag	True if fix is marked junk
geometry	lat/lon	Point geometry (WGS 84)
extra__subject__name	subject.name	Elephant display name
extra__subject__hex	subject.hex_color	Subject colour used in maps

Column	Source field	Description
extra__subject__sex	subject.sex	Subject sex (M / F / unknown)
extra__created_at	observation.created_at	Record creation timestamp
extra__subject__subject_subtype	subject.subject_subtype	Subject subtype (e.g. elephant)

Three known-bad coordinate pairs are filtered out unconditionally:

- (180.0, 90.0) — boundary sentinel
- (0.0, 0.0) — null-island artefact
- (1.0, 1.0) — common default / test value

3.2 Day / Night Annotation

Each relocation is labelled as day or night by classify_is_night. The function calculates solar elevation at the fix's location and UTC timestamp using the **ephem** library. A fix is classified as night (is_night = True) when the sun is below the horizon. This boolean is later used as a colour key in the Day/Night map.

3.3 Trajectory Segment Filter

Before trajectory construction, a **trajectory segment filter** is defined by the user (custom_trajectory_segment_filter). The same filter object is applied to *both* the current period and the previous period trajectories, ensuring methodological consistency in comparisons.

Parameter	Default	Unit	Purpose
min_length_meters	0.001	m	Remove sub-centimetre GPS noise
max_length_meters	5 000	m	Remove implausible long jumps
min_time_secs	1	s	Remove zero-duration segments
max_time_secs	21 600	s	Remove gaps > 6 h (fix dropout)
min_speed_kmhr	0.01	km/h	Remove near-stationary fixes
max_speed_kmhr	9	km/h	Remove biologically impossible speeds for elephants

Note: 9 km/h is used as the elephant speed ceiling because sustained travel above this threshold is not observed in the wild under normal conditions. Segments exceeding this value typically indicate GPS multipath, collar store-and-forward artefacts, or data-entry errors.

3.4 Relocations → Trajectories

Filtered relocations are connected into **linear trajectory segments** by relocations_to_trajectory. Each segment joins two consecutive fixes for the same subject and records:

- **dist_meters** — straight-line distance between the two fixes
- **speed_kmhr** — dist_meters ÷ elapsed time × 3.6
- **segment_start / segment_end** — timestamps of the bounding fixes
- **geometry** — LineString in WGS 84

A temporal index is then added (add_temporal_index) keyed to segment_start and grouped by subject name, enabling efficient per-subject iteration in all downstream tasks.

4. Map Outputs — Methodology

4.1 Movement Tracks Map

The Movement Tracks map overlays **current period** and **previous period** trajectories for each subject, allowing rangers to assess range shifts across time.

How the dual-period display is constructed

After trajectories for both periods are built independently (both filtered by the same segment filter), a duration_status column is appended — 'Current tracks' or 'Previous tracks'. The two GeoDataFrames are merged (merge_multiple_df) and then filtered by filter_groups_by_value_criteria with criteria: priority and priority_value: 'Current tracks'. This step removes subject–periods for which only previous-period data exists but no current data — preventing orphaned previous tracks from appearing for subjects no longer in the group.

Colour assignment

modify_status_colors maps each unique duration_status value to an RGBA colour. Current tracks receive a vivid colour derived from the subject's own hex_color field; previous tracks receive a muted variant. Trajectories are sorted descending by duration_status so current tracks are always rendered on top.

Line styling

Property	Value
Width	2.85 px (min 2, max 8)
Opacity	55 %
Cap / Join	Rounded
Width units	pixels (screen-space, zoom-independent)

4.2 Speed Map

The Speed Map colours each trajectory segment by its average speed, making it easy to identify corridors of fast travel versus slow foraging areas.

Classification

apply_classification bins the speed_kmhr column using the **Equal Interval** scheme with **k = 6** classes. Equal interval divides the observed speed range into six bins of equal width. Labels are formatted to one decimal place with the suffix km/h.

> **Note:** Equal interval is chosen here (rather than natural breaks) to produce classes with intuitive, evenly spaced boundaries that are easy to communicate to non-technical readers.

Colour ramp

Bin (low → high)	Hex colour	Perception
1 (slowest)	#1a9850	Dark green — resting / foraging
2	#91cf60	Light green
3	#d9ef8b	Yellow-green
4	#fee08b	Yellow — moderate travel
5	#fc8d59	Orange
6 (fastest)	#d73027	Red — fast directional movement

This is a diverging green-to-red ramp (adapted from ColorBrewer *RdYlGn*) — perceptually ordered from cool/slow to warm/fast.

4.3 Day / Night Map

Segments are coloured by the is_night boolean computed during relocation annotation (Section 3.2).

Value	Colour	Hex
Day (is_night = 0)	Amber	#e7a553
Night (is_night = 1)	Dark blue	#292965

Trajectories are sorted ascending by is_night so that night segments (value 1) are rendered last and appear on top of day segments where they overlap.

4.4 Home Range Map (ETD + MCP)

The Home Range map combines two complementary estimators: Elliptical Time Density (ETD) contours, which capture the probability density of space use, and a Minimum Convex Polygon (MCP), which shows the total bounding extent of all observed locations.

4.4.1 Elliptical Time Density (ETD)

ETD is a bivariate kernel density estimator that accounts for the temporal autocorrelation inherent in GPS telemetry. Instead of treating each fix as an independent observation, it weights space use by the time spent in each area, producing more ecologically realistic home range estimates.

Key parameters used in calculate_elliptical_time_density:

Parameter	Value	Meaning
CRS	ESRI:53042	World Azimuthal Equidistant — equal-area, minimises distortion for range-wide computat
cell_size	Auto-scale	Cell size is derived automatically from the data extent and trajectory density
max_speed_factor	1.05	Segments with speed > 1.05 × median are down-weighted (reduces motorway artefacts)
expansion_factor	1.3	Raster extent is expanded 30 % beyond the data envelope to capture edge usage
band_count	1	Single density band output
nodata_value	NaN	Cells outside the probability mass are set to NaN (transparent)
percentiles	50, 60, 70, 80, 90, 95, 99.9	Contours at these probability thresholds are extracted as polygons

The resulting raster is reprojected back to **EPSG:4326 (WGS 84)** before visualisation. Contour polygons are coloured using the **RdYlGn** diverging colormap — innermost (50th percentile core) in red, outermost (99.9th percentile) in green — providing an intuitive density gradient.

4.4.2 Minimum Convex Polygon (MCP)

generate_mcp_gdf computes the convex hull of all fix locations in the planar CRS **ESRI:53042** and reports the polygon area in km². The MCP is displayed as a hot-pink outline (RGB 255, 20, 147) with no fill, so the ETD contours remain visible beneath it.

> **Note:** ESRI:53042 (World Azimuthal Equidistant, centred at the equator) is used for all area calculations in this workflow because it preserves distances from the centre point and produces consistent area metrics across the African continent without the distortion of geographic coordinate systems.

4.5 Mean Speed Raster

The Mean Speed Raster aggregates segment-level speeds onto a regular hexagonal grid, revealing landscape-scale patterns in elephant movement intensity.

Raster generation — ecograph parameters

The raster is computed by generate_ecograph_raster (the *ecograph* algorithm from the Ecoscope library):

Parameter	Value	Effect
step_length	User-set (default 2 000 m, or auto)	Trajectories are resampled to this step length before aggregation, regularising the co
movement_covariate	speed	The metric being aggregated is speed (km/h)
interpolation	mean	Within each hex cell, values are averaged across all contributing steps
radius	2	Each hex cell also incorporates contributions from its 2-ring neighbourhood (smoothi
dist_col	dist_meters	Source column for step-length weighting
cutoff	null	No hard cutoff on contributing distance
tortuosity_length	3	Path sinuosity is computed over 3-step windows
resolution	null	Hex cell size is auto-derived from data density
network_metric	null	Network-based movement metrics are not computed for this report

Note: Unlike the other ecograph parameters, **Step Length** is exposed as a user-configurable field in the Mean Speed Raster step of the workflow form — it is not fixed by the spec.

Classification and colouring

The raw per-cell mean speed values are classified using **Natural Breaks (Jenks)** with **k = 6** classes. Natural breaks minimises within-class variance and maximises between-class variance — well-suited to speed rasters where the distribution is typically right-skewed (many slow cells, few fast ones).

The same six-colour green-to-red ramp used on the Speed Map is applied here, so readers can intuitively compare the two products. Labels are formatted to one decimal place in km/h.

Note: The key distinction between the Speed Map and the Mean Speed Raster: the Speed Map colours individual trajectory segments (showing routes), whereas the Mean Speed Raster aggregates all movement across a spatial grid (showing landscape-level patterns). Areas used by fast-moving elephants appear as red hex cells even if no single track crosses them.

4.6 Seasonal Home Range Map

The Seasonal Home Range map shows how elephant space use shifts between the **wet** and **dry** seasons — a key indicator of resource availability and migratory behaviour.

Season determination — GEE NDVI

custom_determine_season_windows queries the Google Earth Engine NDVI time series for the region of interest (derived from the ETD extent) over the analysis period. Wet and dry season windows are identified from the NDVI phenology curve: periods where NDVI exceeds a threshold are classified as *wet*; periods below are classified as *dry*. The function returns a DataFrame of dated season intervals (persisted as a CSV for auditability).

Note: The season boundaries are data-driven — they reflect the actual green-up and senescence pattern for the specific landscape and year, rather than fixed calendar dates. This is critical in East African ecosystems where rainfall timing can vary considerably year to year.

Note: A short analysis period can cause determine_seasonal_windows to fail with EEException('User memory limit exceeded.') — there isn't enough NDVI history for Earth Engine to resolve season windows in one synchronous computation. Widen the workflow's Since/Until range to at least two to three months and re-run.

Season labelling on trajectories

create_seasonal_labels joins the season windows to the trajectory GeoDataFrame on timestamp, adding a season column (e.g. 'wet' or 'dry') to each segment.

ETD calculation per season

calculate_seasonal_home_range groups the labelled trajectories by season and runs an ETD calculation for each season group. Only the **99.9th percentile** contour is extracted, giving a near-total home range boundary per season. Cell size is auto-scaled for each season independently.

Colour assignment

assign_season_colors maps each season label to a fixed colour palette (wet = blue family, dry = orange/brown family). Polygons are rendered as filled, semi-transparent GeoJSON layers (opacity 55 %) so both seasons are visible when overlapping.

4.7 Map Rendering & Tooltips

All six maps are drawn with the built-in **ecoscope-platform** map tasks rather than the earlier copies in ecoscope-workflows-ext-custom: create_path_layer for the track maps (Movement Tracks, Speed, Day / Night), create_geojson_layer for the polygon maps (ETD, MCP, Mean Speed Raster, Seasonal Home Range), and draw_map with output_type: html to render each map as an interactive DeckGL page. Layer and legend styling is unchanged.

Every layer sets tooltip_columns, so hovering over a feature on the interactive map shows these columns:

Layer	Tooltip columns
Movement tracks	duration_status, duration_status_colors, is_night
Speedmap	speed_bins, speed_bins_colormap, speed_kmhr
Day / Night	is_night, day_night_colors
Home range (ETD)	percentile, etd_percentile_colors, area_sqkm
MCP polygon	area_km2
Mean speed raster	value_bins, bins_formatted, speedraster_bins_colormap
Seasonal home range	season, season_colors

Note: The *_colors / *_colormap columns hold the colour each feature is drawn with, so the tooltip shows that raw colour value. Tooltips exist only on the HTML maps; the PNG screenshots in the mapbook are static.

5. Summary Metrics

Three scalar metrics are computed per subject and displayed as widgets in the dashboard and mapbook.

Metric	Source task	How it is calculated
Total MCP Area (km²)	dataframe_column_sum → total_mcp_area	Sums the area_km2 column of the MCP GeoDataFrame (calculated in
Total Grid Area (km²)	dataframe_column_sum → total_grid_area	Sums the area_sqkm column of the ETD GeoDataFrame (grid cell are

Metric	Source task	How it is calculated
Gender	dataframe_column_first_unique_str → subject_gender	Returns the first unique value in the subject_sex column (M, F, or unk

The report duration (in months, rounded to 2 decimal places) is also computed from the analysis time range and included in the mapbook context page.

6. Mapbook Report (.docx)

The final deliverable is a **multi-section Word document** assembled from a cover page and one section per subject.

Both the report logo and an **Include Maps** toggle (default: enabled) are configured under the **Generate Word Doc Report** step. Disabling the toggle skips PNG conversion for all six interactive map types, so the mapbook sections are generated without map images.

6.1 Cover Page

A Word template (mapbook_cover_page.docx) is downloaded from a Dropbox URL and populated with:

- Total subject count
- Analysis time range (since / until)
- Prepared by: Ecoscope
- Organisation logo (user-supplied PNG or JPG)

6.2 Per-Subject Section

For each subject a section template (mapbook_grouper_template.docx) is rendered with:

- Subject name and sex
- Current and previous period dates
- Report duration in months
- MCP area and ETD grid area (km²)
- Six map images (PNG, rendered via headless Playwright screenshot at 2× device scale factor)

Map images are generated by adjust_map_zoom_and_screenshot, which loads each interactive HTML map, adjusts the view state to the pre-computed GDF extent, waits 40 seconds for tiles to load, then captures a full-resolution screenshot. Image boxes are 11.11 × 6.5 cm.

6.3 Document Merge

merge_mapbook_files concatenates the cover page document and all per-subject sections into a single Word file, maintaining correct page ordering and section breaks.

7. Interactive Dashboard

An interactive dashboard is assembled from all widget outputs via gather_dashboard. The dashboard contains:

Widget	Type	Source
Gender	Text	subject_sex first unique value
Total MCP Area	Single value	MCP area in km²

Widget	Type	Source
Total Grid Area	Single value	ETD grid area in km²
Movement Tracks	Map	Interactive DeckGL HTML per subject
Home Range	Map	ETD + MCP DeckGL HTML per subject
Speed Map	Map	Speed-coloured DeckGL HTML per subject
Mean Speed Raster	Map	Ecograph hex raster DeckGL HTML
Day / Night Tracks	Map	Day/night coloured DeckGL HTML
Seasonal Home Range	Map	Season-coloured ETD DeckGL HTML

8. Output Files

All outputs are written to the directory defined by the ECOSCOPE_WORKFLOWS_RESULTS environment variable.

File pattern	Format	Content
trajectories.geoparquet	GeoParquet	Current period trajectory segments with speed bins
previous_period_trajectories.geoparquet	GeoParquet	Previous period trajectory segments
relocations.geoparquet	GeoParquet	Current period GPS fix locations
previous_period_relocations.geoparquet	GeoParquet	Previous period GPS fix locations
<subject>_etd.geoparquet	GeoParquet	ETD contour polygons (WGS 84) per subject
<subject>_mcp.geoparquet	GeoParquet	MCP polygon per subject
<subject>_seasonal_etd.geoparquet	GeoParquet	Seasonal ETD contours per subject
<subject>_ndvi_seasons.csv	CSV	GEE-derived season windows with NDVI values
<subject>_movement_tracks.html	HTML	Interactive movement tracks map
<subject>_speedmap.html	HTML	Interactive speed map
<subject>_day_night.html	HTML	Interactive day/night map
<subject>_homerange.html	HTML	Interactive home range map
<subject>_mean_speed_raster.html	HTML	Interactive mean speed raster map
<subject>_seasonal_home_range.html	HTML	Interactive seasonal home range map
mapbook_context_page.docx	Word	Mapbook cover page
<subject>.docx	Word	Per-subject mapbook section
<merged_mapbook>.docx	Word	Final combined mapbook

9. Workflow Execution Logic

9.1 Grouping Strategy

The grouper is set to subject_name (the individual elephant's display name from EarthRanger). All map and metric tasks that use mapvalues iterate over the split-by-group data — producing one output per subject.

9.2 Skip Conditions

Every task in the workflow has two default skip conditions:

- **any_is_empty_df** — if any input DataFrame is empty, the task and all its dependants are skipped rather than raising an error.
- **any_dependency_skipped** — if an upstream task was skipped, all downstream tasks that depend on it are also skipped automatically.

Widget creation tasks additionally have skipif: conditions: [never], meaning they always run even if their input data would normally trigger a skip — this ensures the dashboard assembles correctly with partial data.

9.3 Previous Period Logic

The previous period is determined by flexible_previous_period. In every mode, the comparison period's **End Date** is fixed to the current time range's Start Date (so the two periods never overlap) — only the Start Date calculation differs. There are three modes:

Mode	Behaviour
Preset	Choose a common lookback — Same as current period, 1 month back, 3 months back, 6 months back, or 1 ye
Calendar	Manually pick the exact Start Date for the comparison period
Custom	Enter a Years / Months / Weeks / Days offset to count backward from the current time range's start date (defau

Default is the **Preset** mode with **Same as current period** selected (set in the spec's form overrides). It subtracts the length of the current time range from its start date, giving a comparison period of equal length immediately before it — e.g. a 1 Jul – 30 Sep range is compared with 1 Apr – 1 Jul. The fixed presets subtract a calendar offset (relativedelta) from the start date instead.

10. Software Versions

As of the migration to the **ecoscope-platform** task/compiler runtime, the workflow's requirements are:

Package	Version	Role
ecoscope-platform	==2.25.3	Core task library and workflow engine, including the map layer, draw
ecoscope-workflows-ext-custom	0.1.4.*	Custom STE utility tasks
ecoscope-workflows-ext-ste	0.0.24.*	STE-specific tasks (mapbook, seasonal analysis, map overlay, previ
pydeck	0.9.2	Renders the interactive DeckGL maps
opentelemetry-sdk	>=1.20.0, <2.0.0	Workflow run telemetry/observability instrumentation

Note: ecoscope-workflows-ext-ecoscope and ecoscope-workflows-ext-big-life are no longer direct dependencies — their task coverage now lives in ecoscope-platform.

ecoscope-platform is pulled from the ecoscope-workflows prefix.dev channel; the two ecoscope-workflows-ext-* packages from the ecoscope-workflows-custom channel; pydeck and opentelemetry-sdk from conda-forge. The runtime environment is managed by **pixi**.